AI-Driven Network Traffic Analysis and Intrusion Detection System

Technical Report

Practical Technical Assessment

Cybersecurity & Network Security Research Intern

MegaMinds IT Services

Prepared by	Ifrah Hasware
Repository	`megaminds-ids`
Report length	1,898 words (excluding tables)
Date	24 October 2026

Contents

1. Problem overview 1
2. Objectives 1
3. Traffic data and dataset 1
4. System architecture 2
5. Feature extraction methodology 2
6. Detection methodology 3
7. Testing and evaluation 3
8. Results 3
9. Limitations 4
10. Future improvements 5

1. Problem overview

Signature-based intrusion detection compares traffic against a catalogue of known-bad patterns. It is precise, fast and trivially explainable, but it is structurally blind to anything not already in the catalogue: a new scanner variant, a re-tuned flood rate, or a slow probe designed to stay under per-packet thresholds will pass through untouched. Behavioural and machine learning approaches invert that trade-off. They generalise to unseen traffic but produce opaque verdicts and higher false-positive rates, and an analyst cannot act on a score of 0.87 without knowing which behaviour drove it.

The problem this project addresses is therefore not merely "detect attacks". It is to build a detector that is simultaneously **generalising** and **accountable** — one where every flag can be justified to a human in terms they can verify, and where the two detection philosophies are combined so that each covers the other's blind spot. A label with no supporting explanation is explicitly not sufficient.

2. Objectives

1. Produce traffic containing benign activity and four attack families: reconnaissance, denial of service, credential brute force, and a man-in-the-middle variant, with exact per-flow ground truth.
2. Extract features that describe *behaviour over time* rather than individual packets, so that a scan is distinguishable from a busy client.
3. Build a detection engine whose decisions are reproducible and whose thresholds are inspectable.
4. Make every alert self-explanatory: what fired, why, with what confidence.
5. Quantify performance honestly, including the cases the system gets wrong.

3. Traffic data and dataset

Captures are synthesised with Scapy (`data/generate_dataset.py`) rather than recorded from a live network. Three considerations drove that decision.

First, **ground truth is exact**. Because the generator crafts each packet, the label of every flow is known by construction, including the precise boundary between an attack and its surrounding benign background. Public datasets such as CICIDS2017 provide labels at a coarser granularity and cannot be extended with a new attack variant without re-labelling by hand.

Second, **reproducibility**. A single seed regenerates byte-identical traffic on any machine. A reviewer can re-run the entire evaluation and obtain the same numbers, which is not true of a live capture.

Third, **no privileged access**. Live capture on Windows requires Npcap and administrator rights, and cannot run in CI. The pipeline must be runnable by the evaluator without special setup.

The trade-off is realism, and it is a real one: the traffic is structurally valid — correct three-way handshakes, monotonic sequence numbers, legal flag semantics, plausible payload sizes — but it is not a real network. There is no genuine latency jitter, no retransmission behaviour, no background noise from unrelated protocols. Absolute performance figures should therefore be read as a measure of *internal consistency and methodology*, not as a production benchmark.

The topology models a small enterprise: gateway `10.0.0.1`, DNS `10.0.0.53`, web server `10.0.0.10`, database `10.0.0.11`, seven client hosts, a dedicated audit station `10.0.0.200`, and external destinations reached through the gateway.

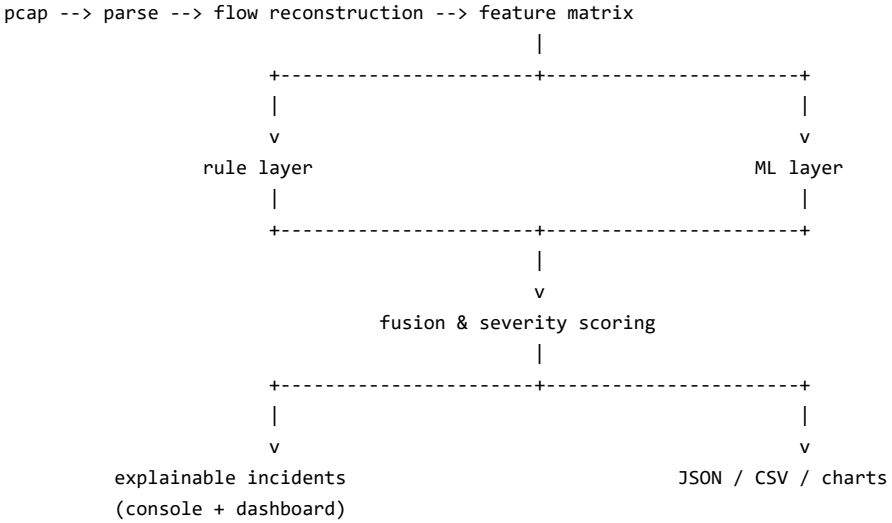
Six captures were produced, totalling **9,776 packets and 2,844 flows**:

Capture	Flows	Content
`scenario_1_benign`	71	HTTP, HTTPS, DNS, admin SSH, ICMP, NTP, MySQL — no attacks
`scenario_2_port_scan`	243	120-port vertical SYN scan, 40-host horizontal sweep, NULL and XMAS scans
`scenario_3_dos`	2,257	distributed SYN flood (60 spoofed sources), UDP flood, ICMP flood
`scenario_4_brute_force`	90	18 failed SSH logins, 12 failed RDP logins
`scenario_5_ambiguous`	130	authorised audit scan, deliberately slow scan, 1,800-packet backup
`scenario_6_mitm_arp`	53	ARP poisoning of the gateway and web server

Scenario 5 exists specifically to be difficult. It contains an *authorised* vulnerability scan that is indistinguishable from reconnaissance at the packet level, a scan spread at one probe every three seconds so that no 10-second window ever crosses the threshold, and a legitimate 1,800-packet backup whose throughput resembles a flood.

4. System architecture

The pipeline is a linear sequence of pure functions, which keeps each stage independently testable:



Capture parsing streams frames with `PcapReader` so memory stays flat, then folds them into bidirectional 5-tuple flows; a flow is closed after 60 seconds of silence so a long-lived service that is periodically reused does not become one enormous flow. ARP has no IP 5-tuple and is handled on a separate layer-2 path.

5. Feature extraction methodology

Fifty-five features are computed in three groups.

Per-flow volume and timing: duration, forward/backward packet and byte counts, packet and byte rate, payload ratio, packet-size mean/std/min/max per direction, and inter-arrival time statistics. Automated traffic is metronomic and uses uniformly small packets; user traffic is bursty and bimodal. These features make that difference measurable.

TCP flag behaviour: counts and ratios for SYN, ACK, FIN, RST, PSH and URG, plus three derived booleans — `handshake_completed`, `failed_handshake` (a SYN that never received a SYN/ACK) and the illegal flag combinations `null_scan` (no flags at all) and `xmas_scan` (FIN+PSH+URG). `failed_handshake` is the single most discriminative feature in the system: a scan is by definition a stream of connections that never complete.

Host behaviour over a sliding 10-second window: how many distinct hosts and distinct ports

a source touched, the **Shannon entropy of its destination port distribution**, the number of unanswered probes it emitted, the number of login attempts it made, and — from the victim's perspective — how many distinct sources converged on a single destination. Fan-out indicates scanning; fan-in indicates a distributed flood. This is what allows the system to recognise a *pattern across flows*, which no single-flow feature can express.

Two decisions deserve justification. Rates are reported as **peak packets-per-second** rather than window averages: a 300-packet burst inside one second of a ten-second window averages to 30 pps and would slip under every flood threshold. And port entropy is preferred to a raw port count because a client legitimately touching twenty ports of one service has low entropy while a scanner sweeping 120 ports has entropy $\approx \log_2(120) \approx 6.9$; entropy encodes the *diversity* of the sweep, not merely its size.

For the ML stage, heavy-tailed counters are `log1p`-compressed so a 100 MB transfer cannot dominate every distance metric, and the raw `dst_port` integer is excluded — otherwise the model would learn that port 3389 is "a big number" rather than learning behaviour.

6. Detection methodology

Signature layer. Nine deterministic rules (R-001 … R-008) encode the network-security invariants directly. A vertical scan is $\geq$15 distinct ports from one source in ten seconds with $\geq$5 unanswered probes; a horizontal scan is $\geq$10 distinct hosts on $\leq$3 ports; a SYN flood is either $\geq$40 distinct sources converging on one victim at $\geq$150 pps, or $\geq$100 pps of 80%-SYN traffic from a single source; brute force is $\geq$8 short-lived sessions to port 22 or 3389; ARP poisoning is one IP claimed by two or more MAC addresses. Every threshold lives in `nids/config.py` and can be overridden from the command line, which is what makes the sensitivity analysis in Section 8 possible.

Machine learning layer. An Isolation Forest is trained on benign flows only, so it can in principle flag behaviour no rule was written for. A Random Forest is trained on labelled flows with balanced class weights. The classifier is the primary ML signal; the Isolation Forest stands alone only when its calibrated score exceeds 0.90, a threshold chosen because no benign flow in the baseline reaches it.

Fusion. Flow-level hits are collapsed into one incident per (rule, entity, time window), so a 120-port scan is one alert carrying 120 flows rather than 120 alerts. Agreement between the layers raises severity; **disagreement never lowers it**. A rule is deterministic evidence — 120 unanswered SYNs really did occur — and allowing a probabilistic model to demote that would be a safety regression. Disagreement is instead expressed through a reduced confidence score and an explicit sentence in the alert text.

7. Testing and evaluation

Scenarios 1–4 form the development set and are split with `GroupShuffleSplit` keyed on `(scenario, time window)`, so an entire incident and its surrounding benign background land on one side of the split. A per-flow random split would place near-duplicate scan probes in both train and test and inflate every metric; grouping by window prevents that leakage. Scenarios 5 and 6 are never used for training and serve as pure hold-out data.

The system is scored at **flow level** — did it flag this conversation? — because that is the granularity at which labels exist. Alert-level precision is reported separately because that, not flow-level accuracy, determines analyst workload. Each of the three configurations is scored on identical flows.

8. Results

Layer	TP	FP	TN	FN	Precision	Recall	F1	Accuracy
Signature only	2,403	40	375	26	0.984	0.989	0.987	0.977
ML only	2,411	41	374	18	0.983	0.993	0.988	0.979
Hybrid	**2,428**	**40**	**375**	**1**	**0.984**	**1.000**	**0.992**	**0.986**

Attack class	Flows	Signature recall	ML recall	Hybrid recall
Port scan	197	0.873	1.000	1.000
Denial of service	2,201	1.000	1.000	1.000
Brute force	30	1.000	0.400	1.000
MITM (layer-2)	—	see §8.3		

Twelve alerts were raised in total: eleven true positives and one false positive, an alert-level precision of 0.917.

8.1 Handled well. The benign baseline produced **zero alerts** across 71 flows. Both scans, all three floods and both brute-force campaigns were detected as single consolidated incidents rather than alert storms — the SYN flood reduced 1,800 attacking flows to one critical alert identifying the victim, the 60 distinct sources and the peak rate.

8.2 The clearest argument for a hybrid. On the slow scan, the signature layer achieved 0.873 recall on port scans generally but **zero** on this attack: one probe every three seconds never crosses the 15-port window threshold. The Random Forest, which reasons over per-flow shape rather than window counts, scored those probes at an attack probability of 0.97 and raised a single ML-only alert covering 25 flows. That is precisely the failure mode a hand-written rule set cannot cover and a learned model can.

8.3 Layer-2 caveat. ARP poisoning produces no IP flow of its own, so it is scored at the event level, not through the flow confusion matrix. Both ARP events — the gateway's IP claimed by the attacker's MAC, and the web server's — were detected as CRITICAL. The single `mitm` flow label in the table is the victim's redirected HTTP conversation, which is indistinguishable from ordinary traffic once the cache is poisoned; counting it as a flow-level false negative understates MITM detection.

8.4 False positives. All 40 false-positive flows come from scenario 5, and all of them are the authorised audit scan. On the five non-ambiguous scenarios the false-positive rate is **zero**. The remaining false positive is the honest cost of a threshold that cannot distinguish an audit from an intrusion without an allow-list.

8.5 False negatives. The signature layer missed 26 flows (the slow scan). The ML layer caught those but missed 18 brute-force flows, where the classifier had too few training exemplars after the group split. The hybrid missed one.

8.6 Sensitivity. Because every threshold is a CLI parameter, the precision/recall trade-off is reproducible: lowering `--scan-ports` to 5 catches the slow scan with the rule layer but also flags eight-port clients; raising it to 40 loses the vertical scan entirely.

9. Limitations

The dataset is synthetic, so absolute metrics will not transfer unchanged to production. The Isolation Forest under-performs here — attack flows average 0.74 against 0.52 for benign, far too close to carry a decision alone — because the discriminating signals are strongly behavioural and the supervised layer learns them directly; IF is retained as a second opinion rather than a primary detector. Detection stops at the transport layer, so TLS-encrypted brute force or exfiltration is invis-

ible. ARP analysis is capture-local and sees only one segment. No inline throughput measurement was performed: the architecture is batch-oriented and would need a ring buffer, sampling and a lock-free hand-off to run inline at line rate, where per-packet latency would also rise.

10. Future improvements

Allow-listing known scanners would remove the audit false positive without weakening R-001. Per-entity adaptive baselining — a window threshold that scales with each host's own historical fan-out — would replace global constants. Adding Zeek for protocol parsing would extend coverage to TLS metadata, SMB and DNS tunnelling. A grouped train/test split at the *attack instance* level across more seeds would tighten the confidence intervals on every number reported here. Finally, converting the batch pipeline to a streaming one (Kafka or a ring buffer) with measured throughput and p99 latency would answer the inline-performance question this report leaves open.